

3G, 4G, GSM, UMTS, Wi-Fi, Bluetooth, ZigBee, etc.

Service management layer: This layer enables IoT application programmers to work with heterogeneous objects, irrespective of the hardware platform.

Application layer: This enables high-quality smart services to fetch what the customers need, as per their requirements. It covers smart homes, smart production units, transportation, smart health care based biosensor equipment, etc.

Business layer: This layer manages the overall IoT system's activities and services. It is responsible for building the business model, graphs and flowcharts on the basis of data acquired at the application layer.

IoT protocols

IEEE (Institute of Electrical and Electronics Engineers) and ETSI (European Telecommunications Standards Institute) have defined some of the most important protocols for IoT. These are listed below.

CoAP (Constrained Application Protocol): This was created by the IETF Constrained RESTful Environments (CoRE) working group. CoAP is an Internet application protocol for constrained devices. It is designed to be used between devices on the same constrained network, between devices and general nodes on the Internet, and between devices on different constrained networks—both joined on the Internet. This protocol is especially designed for IoT systems based on HTTP protocols. CoAP makes use of the UDP protocol for lightweight implementation. It also makes use of RESTful architecture, which is very similar to the HTTP protocol. It is used within mobiles and social network based applications and eliminates ambiguity by using the HTTP

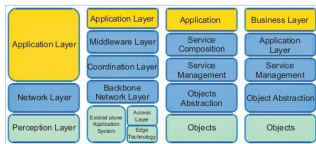


Figure 1: IoT layered architecture

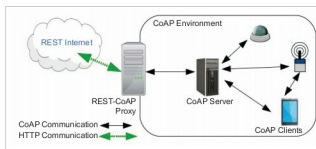


Figure 2: How CoAP works

get, post, put and delete methods. Apart from communicating IoT data, CoAP has been developed along with DTLS for the secure exchange of messages. It uses DTLS for the secure transfer of data in the transport layer.

MQTT Protocol: MQTT (Message Queue Telemetry Transport), a messaging protocol, was developed by Andy Stanford-Clark of IBM and Arlen Nipper of Arcom in 1999. It is mostly used for remote monitoring in IoT. Its primary task is to acquire data from many devices and transport it to the IT infrastructure. MQTT connects devices and networks with applications and middleware. A hub-and-spoke architecture is natural for MQTT. All the devices connect to data concentrator servers like IBM's new MessageSight appliance. MQTT protocols work on top of TCP to provide simple and reliable streams of data.

MQTT Protocol consists of three main components: subscriber, publisher and broker. The publisher generates the data and transmits the information to subscribers through the broker. The broker ensures security by cross-checking the authorisation of publishers and subscribers.

MQTT Protocol is the preferred option for IoT based devices, and is able to provide efficient information-routing functions to small, cheap, low-memory and power-consuming devices in vulnerable and low bandwidth based networks.

XMPP (Extensible Messaging and Presence Protocol): This is a communication IoT protocol for message-oriented middleware based on the XML language. It enables the real-time exchange of structured yet extensible data between any two or more network entities. The protocol was developed by the Jabber open source community in 1999, basically for real-time messaging, presence information, and the maintenance of contact lists. XMPP enables messaging applications to attain authentication, access control, hop-by-hop and end-to-end encryption. Being a secure protocol, it sits on top of core IoT protocols and connects the client to the server via a stream of XML stanzas. The XML stanza has three main components: message, presence and IQ.

AMQP (Advanced Message Queuing Protocol): This was developed by John O'Hara at JPMorgan Chase in London. AMQP is an application layer protocol for message-oriented middleware environments. It supports reliable communication via message delivery assurance primitives

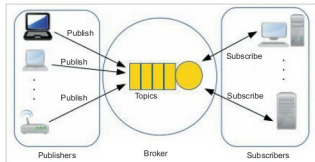


Figure 3: MQTT Protocol architecture